

Lecture# 9

Shared Preferences

By

Dr. Muhammad Bilal

Shared Preferences

- In Android, there are 5 main ways to store data:
 - Shared Preferences – private key-value pair storage
 - Internal Storage – private storage per-application
 - External Storage – local shared storage (e.g., on SD card)
 - SQLite Database – private structured data
 - Network Storage – store data on your own server
- Shared Preferences allows activities and applications to keep preferences, in the form of key-value pairs similar to a Map that will persist even when the user closes the application.
- As the name suggests, the primary purpose is to store user-specified configuration details, such as user specific settings, keeping the user logged into the application.
- Android stores Shared Preferences settings as XML file in shared_prefs folder under **DATA/data/{application package}/shared_prefs/<pref filename>.xml**

Access Shared Preferences

- To get access to the preferences, we have two APIs to choose from:
 - **getPreferences()** : used from within your Activity, to access activity-specific preferences
 - **getSharedPreferences()** : used from within your Activity (or other application Context), to access application-level preferences

Modes of Shared Preferences

Sr.No	Mode & description
1	MODE_APPEND This will append the new preferences with the already existing preferences
2	MODE_PRIVATE By setting this mode, the file can only be accessed using calling application
3	MODE_WORLD_READABLE This mode allow other application to read the preferences
4	MODE_WORLD_WRITEABLE This mode allow other application to write the preferences

Saving & Reading from Shared Preferences

- Storing
 - Call `edit()` to get a `SharedPreferences.Editor`.
 - Add values with editor “put” methods such as `putBoolean()` and `putString()`.
 - Commit the new values with `apply()` or `commit()`.
- Reading
 - Use `SharedPreferences` “get” methods such as `getBoolean()` and `getString()`.
 - The “get” methods have two parameters
 - The preference key string
 - A default value to return if the preference is undefined

Differences Between Methods `apply()` and `commit()`

- `commit()`
 - Returns a boolean value to indicate if the new values were successfully written to persistent storage.
 - Writes its preferences out to persistent storage synchronously (can block the UI thread)
- `apply()`
 - Commits its changes to the in-memory `SharedPreferences` immediately but starts an asynchronous commit to disk.
 - does not block the UI thread, but you won't be notified of any failures.

Methods for Saving & Reading Shared Preferences

// in interface SharedPreferences.Editor

void apply()

boolean commit()

SharedPreferences.Editor putBoolean(String key, boolean value)

SharedPreferences.Editor putFloat(String key, float value)

SharedPreferences.Editor putInt(String key, int value)

SharedPreferences.Editor putLong(String key, long value)

SharedPreferences.Editor putString(String key, String value)

SharedPreferences.Editor putStringSet(String key, Set<String> values)

SharedPreferences.Editor remove(String key)

// in interface SharedPreferences

boolean getBoolean(String key, boolean defValue)

float getFloat(String key, float defValue)

int getInt(String key, int defValue)

long getLong(String key, long defValue)

String getString(String key, String defValue)

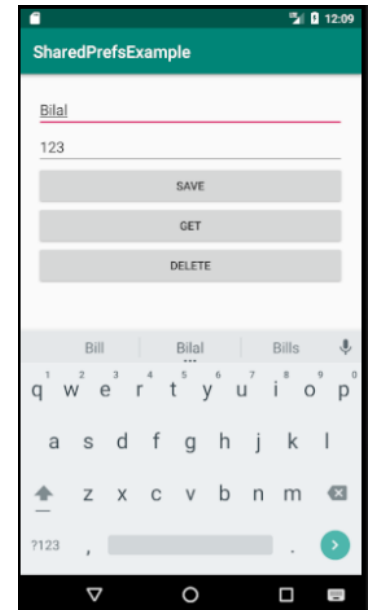
Set<String> getStringSet(String key, Set<String> defValues)

Initializing Shared Preferences

- A good approach is to initialize the shared preferences in new class and implement storing and saving method there.

```
SharedPreferences prefs = context.getSharedPreferences("MyPrefs", context.MODE_PRIVATE);  
SharedPreferences.Editor editor = prefs.edit();
```

- Let us take a example for user name and password, which can save, restore and delete user name and password.



```

public class MySharedPrefs {
    public static final String USER_NAME = "userName";
    public static final String PASS_WORD = "passWord";

    public static boolean setUserNamePW(Context context, String userName, String passWord){
        SharedPreferences prefs = context.getSharedPreferences("MyPrefs", context.MODE_PRIVATE);
        SharedPreferences.Editor editor = prefs.edit();

        editor.putString(USER_NAME, userName);
        editor.putString(PASS_WORD, passWord);
        return editor.commit();
    }

    public static String[] getUserNamePW(Context context){
        SharedPreferences prefs = context.getSharedPreferences("MyPrefs", context.MODE_PRIVATE);

        String[] userNamePW = new String[2];
        userNamePW[0] = prefs.getString(USER_NAME, "");
        userNamePW[1] = prefs.getString(PASS_WORD, "");
        return userNamePW;
    }

    public static boolean removeUserNamePW(Context context){
        SharedPreferences prefs = context.getSharedPreferences("MyPrefs", context.MODE_PRIVATE);
        SharedPreferences.Editor editor = prefs.edit();

        editor.remove(USER_NAME);
        editor.remove(PASS_WORD);
        return editor.commit();
    }
}

```

MySharedPrefs.java

MainActivity.java

```
public class MainActivity extends AppCompatActivity {

    EditText userName, password;
    Button save, restore, delete;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        userName = findViewById(R.id.user_name);
        password = findViewById(R.id.pass_word);
        save = findViewById(R.id.save_btn);
        restore = findViewById(R.id.restore_btn);
        delete = findViewById(R.id.delete_btn);
    }

    public void saveButton(View view){
        String userNameStr = userName.getText().toString();
        String passWordStr = password.getText().toString();
        if(!userNameStr.isEmpty() &&
            !passWordStr.isEmpty()){
            if(MySharedPreferences.setUserNamePW(this, userNameStr, passWordStr)){
                Toast.makeText(this, "Successfully Saved", Toast.LENGTH_SHORT).show();
            } else {
                Toast.makeText(this, "Not Successfully Saved", Toast.LENGTH_SHORT).show();
            }
        }
    }

    public void restoreButton(View view){
        String[] userNamePW = MySharedPreferences.getUserNamePW(this);
        if(!userNamePW[0].isEmpty() && !userNamePW[0].isEmpty()){
            userName.setText(userNamePW[0]);
            password.setText(userNamePW[1]);
        }
    }

    public void deleteButton(View view){
        if(MySharedPreferences.removeUserNamePW(this)){
            userName.setText("");
            password.setText("");
            Toast.makeText(this, "Successfully Deleted", Toast.LENGTH_SHORT).show();
        } else {
            Toast.makeText(this, "Not Successfully Deleted", Toast.LENGTH_SHORT).show();
        }
    }
}
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="20dp"
    tools:context=".MainActivity">

    <EditText
        android:id="@+id/user_name"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:ems="10"
        android:inputType="textPersonName"
        android:text="User Name" />

    <EditText
        android:id="@+id/pass_word"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:ems="10"
        android:inputType="textPersonName"
        android:text="PassWord" />

    <Button
        android:id="@+id/save_btn"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:onClick="saveButton"
        android:text="Save" />

    <Button
        android:id="@+id/restore_btn"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:onClick="restoreButton"
        android:text="Get" />

    <Button
        android:id="@+id/delete_btn"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:onClick="deleteButton"
        android:text="Delete" />

</LinearLayout>
```



Demo

